University of Brescia, Italy
Machine Learning & Data Mining

Overfitting in Decision Trees

Alfonso E. Gerevini

Noisy Training Data in Decision Trees

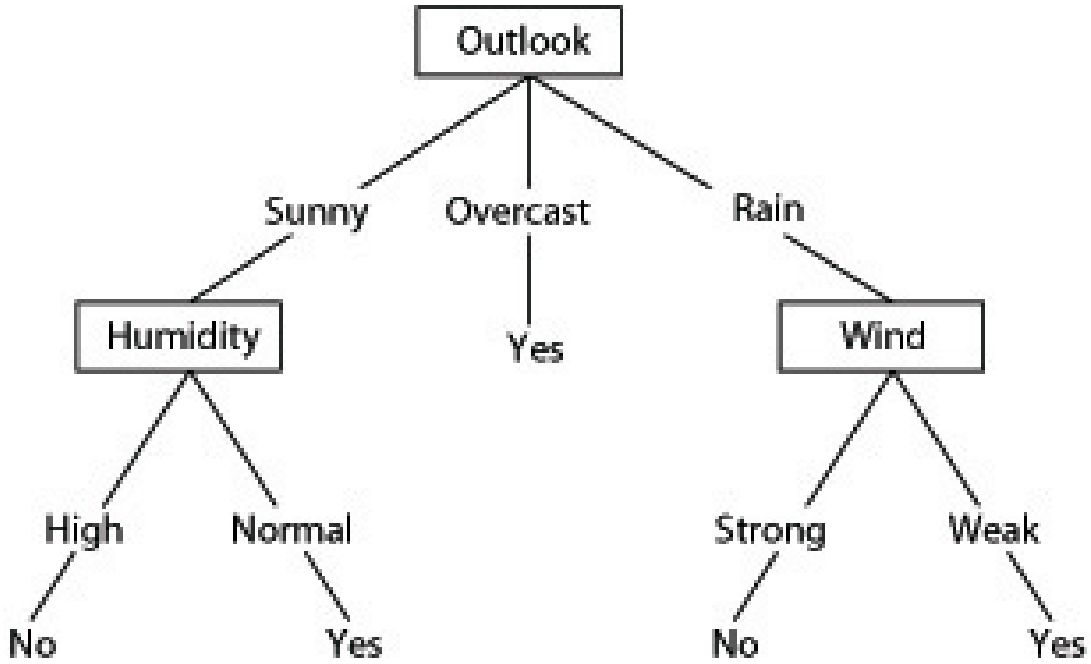
Consider adding a noisy (incorrect) training example

$$D15 = \langle Sunny,\ Hot,\ Normal,\ Strong,\ PlayTennis = No \rangle$$

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Learning a DT with Noisy Data

What effect on earlier tree?



Overfitting

Consider error of hypothesis h over

- training data: $error_{train}(h) \leftarrow$ **training error**
- entire distribution $\mathcal{D}$ of data: $error_{\mathcal{D}}(h) \leftarrow$ **generalisation error**

Hypothesis $h \in H$ **overfits** training data if there is an alternative hypothesis $h' \in H$ such that

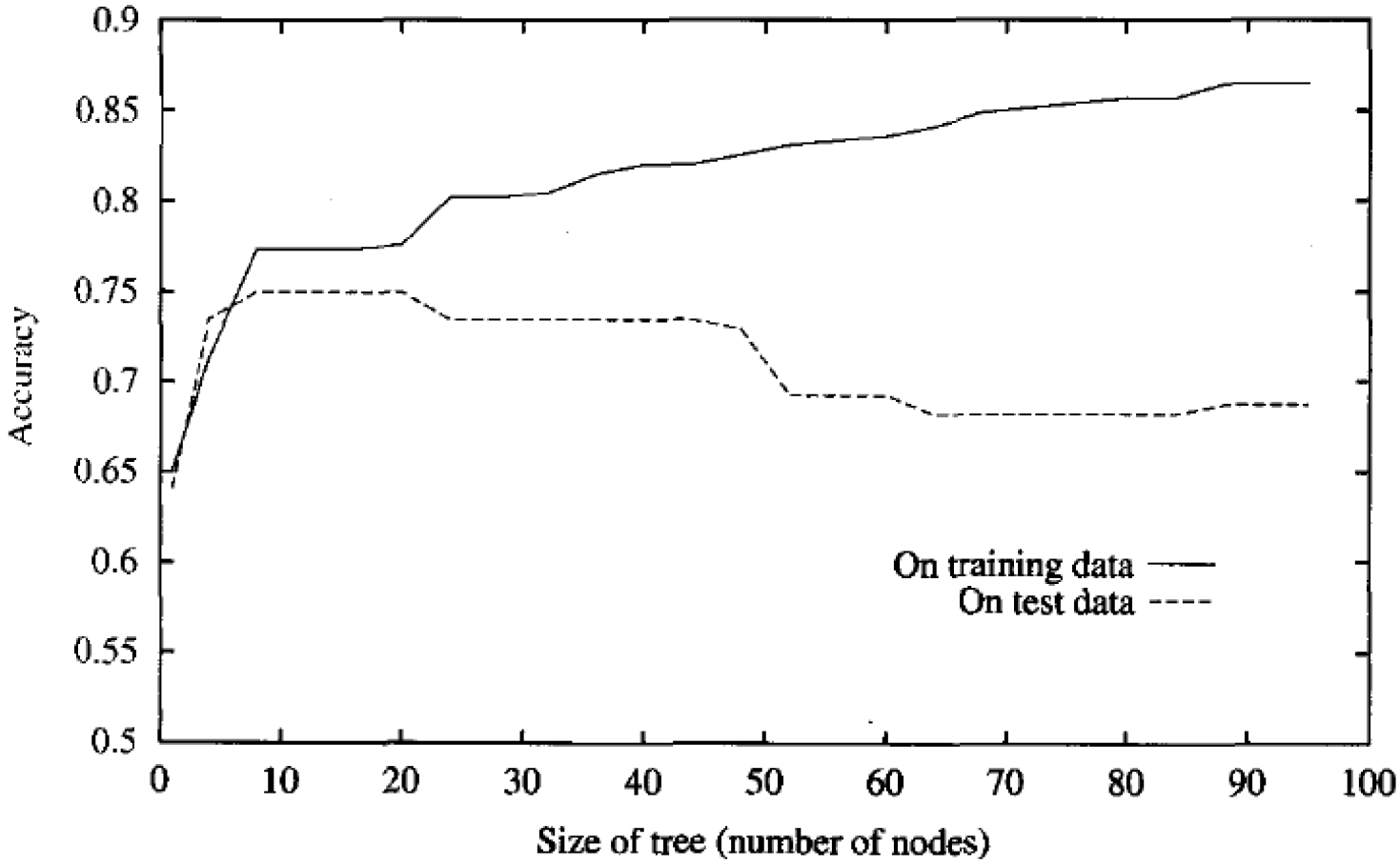
$$error_{train}(h) < error_{train}(h') \quad \wedge \quad error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$$

$$accuracy_{train}(h) > accuracy_{train}(h') \quad \wedge \quad accuracy_{\mathcal{D}}(h) < accuracy_{\mathcal{D}}(h')$$

Can happen when training set too small (underfitting) or contains errors

Overfitting in Decision Tree Learning

Test data set: set of $\langle x_i, Target(x_i) \rangle$ disjoint from training set
Used to measure accuracy of the learned decision tree



Overfitting Example: Noisy Data

Table 4.3. An example training set for classifying mammals. Class labels with asterisk symbols represent mislabeled records.

Name	Body Temperature	Gives Birth	Four-legged	Hibernates	Class Label
porcupine	warm-blooded	yes	yes	yes	yes
cat	warm-blooded	yes	yes	no	yes
bat	warm-blooded	yes	no	yes	no*
whale	warm-blooded	yes	no	no	no*
salamander	cold-blooded	no	yes	yes	no
komodo dragon	cold-blooded	no	yes	no	no
python	cold-blooded	no	no	yes	no
salmon	cold-blooded	no	no	no	no
eagle	warm-blooded	no	no	no	no
guppy	cold-blooded	yes	no	no	no

Table 4.4. An example test set for classifying mammals.

Name	Body Temperature	Gives Birth	Four-legged	Hibernates	Class Label
human	warm-blooded	yes	no	no	yes
pigeon	warm-blooded	no	no	no	no
elephant	warm-blooded	yes	yes	no	yes
leopard shark	cold-blooded	yes	no	no	no
turtle	cold-blooded	no	yes	no	no
penguin	cold-blooded	no	no	no	no
eel	cold-blooded	no	no	no	no
dolphin	warm-blooded	yes	no	no	yes
spiny anteater	warm-blooded	no	yes	yes	yes
gila monster	cold-blooded	no	yes	yes	no

Overfitting Example: Noisy Data

How are the Human and Dolphin test examples classified? Why??

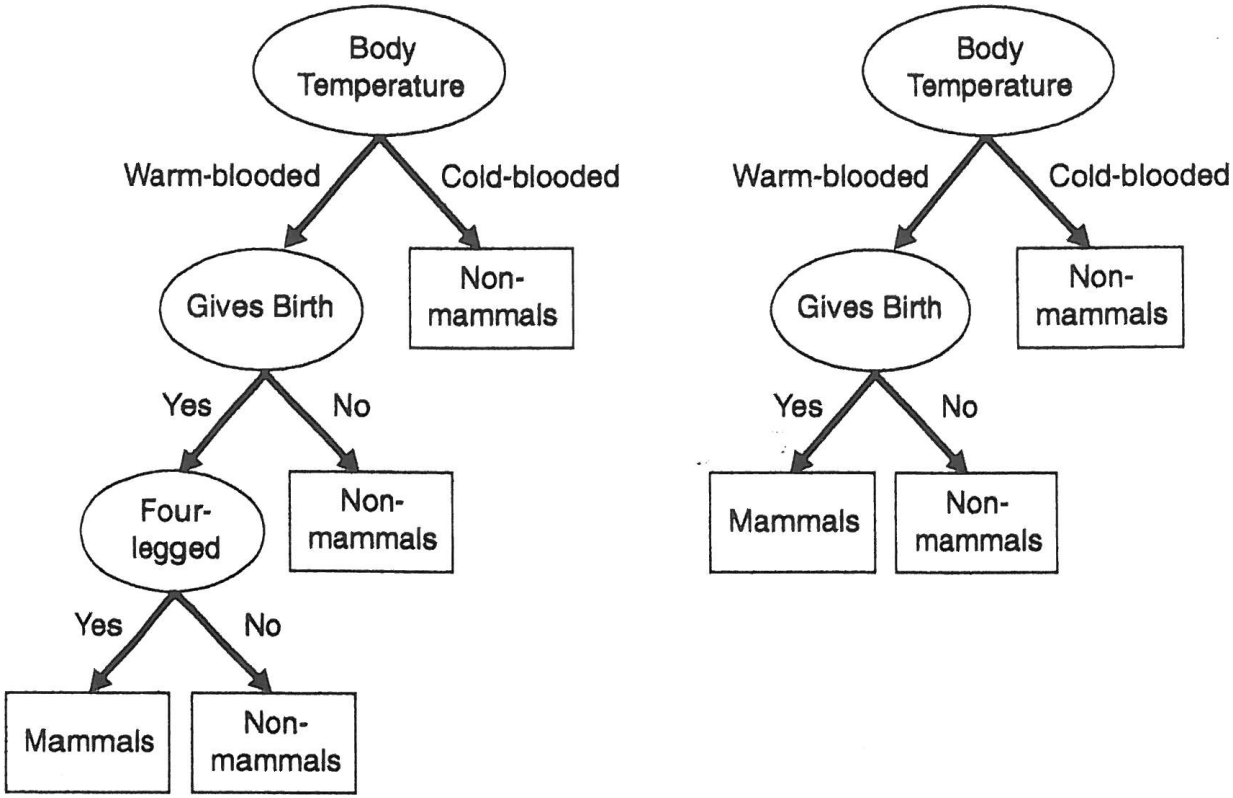


Figure 4.25. Decision tree induced from the data set shown in Table 4.3.

Overfitting Example: Few Representative Data

Table 4.5. An example training set for classifying mammals.

Name	Body Temperature	Gives Birth	Four-legged	Hibernates	Class Label
salamander	cold-blooded	no	yes	yes	no
guppy	cold-blooded	yes	no	no	no
eagle	warm-blooded	no	no	no	no
poorwill	warm-blooded	no	no	yes	no
platypus	warm-blooded	no	yes	yes	yes

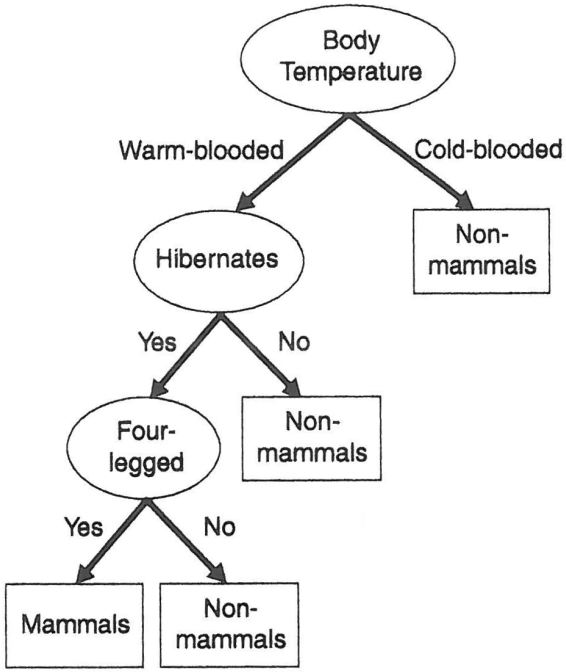


Figure 4.26. Decision tree induced from the data set shown in Table 4.5.

Avoiding Overfitting

How can we avoid overfitting?

- stop growing before perfect classification (when data split not statistically significant)
- grow full tree, then post-prune (practically more successful)

To determine the correct tree size:

1. use a separate set of examples (distinct from the training examples) to evaluate the utility of post-pruning: **validation set**
2. apply a **statistical test** to estimate accuracy of a tree on the *entire data distribution*
3. use an explicit **measure of the complexity** for encoding the examples and the decision trees (halting when size encoding is minimized)

Reduced-Error Pruning

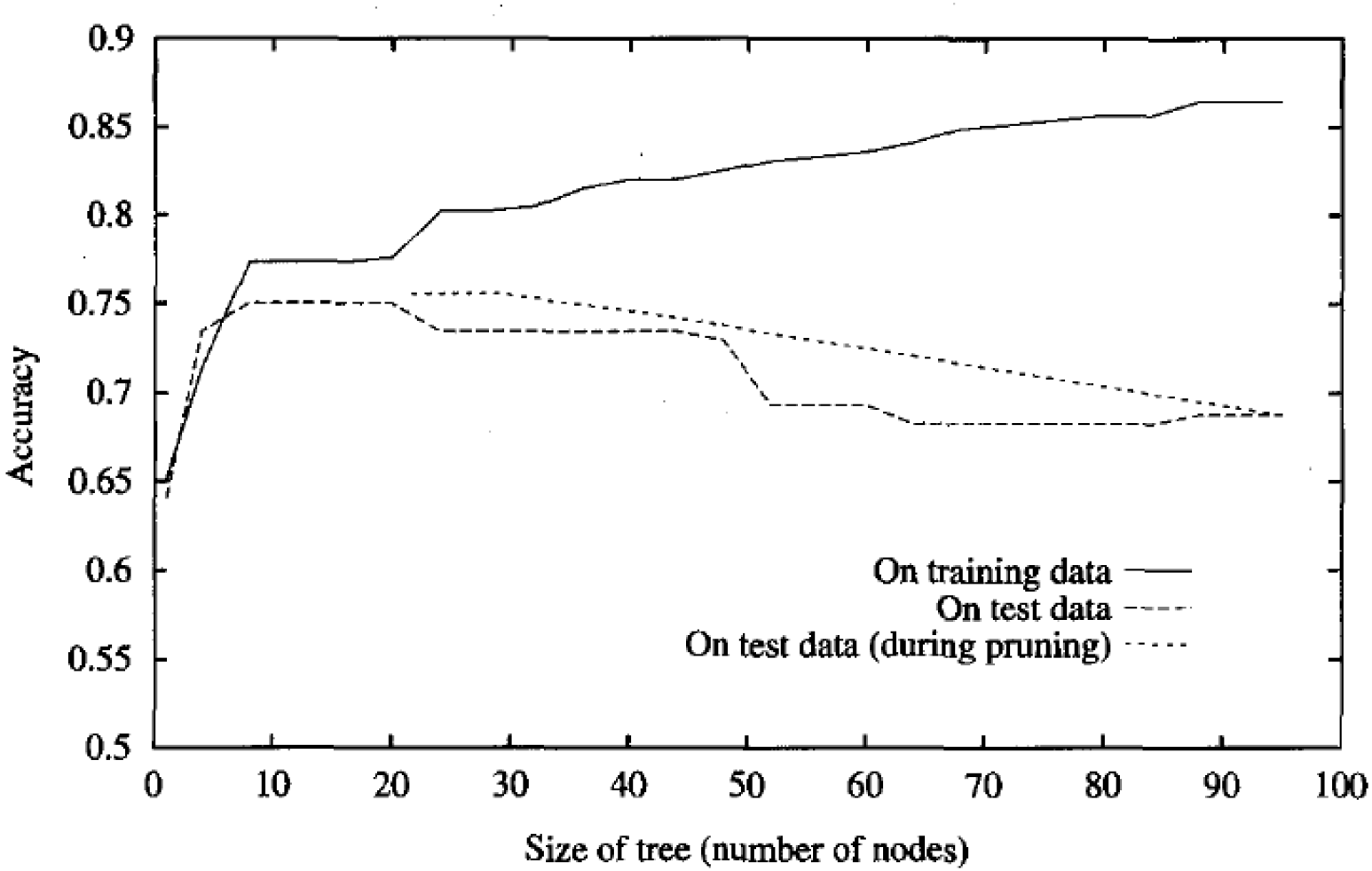
Approach (1) is called *training and validation set* and *Reduced-Error Pruning* is one of the techniques to implement it:

Split data into *training* and *validation* sets ($\neq$ test set!). 3 sets in total

Do until further pruning is harmful (decreases accuracy):

1. Evaluate impact on *validation* set of pruning each possible node (remove all the subtree and assign the most common classification of the associated examples)
2. Greedily remove the one that most improves *validation* set accuracy

Effect of Reduced-Error Pruning



Remarks on Reduced-Error Pruning

- It produces the smallest version of the most accurate subtree (removing sub-trees added due to coincidental irregularities).
- Does not work well when data set is limited:
  reducing the set of training examples (used as validation examples) can give bad results.

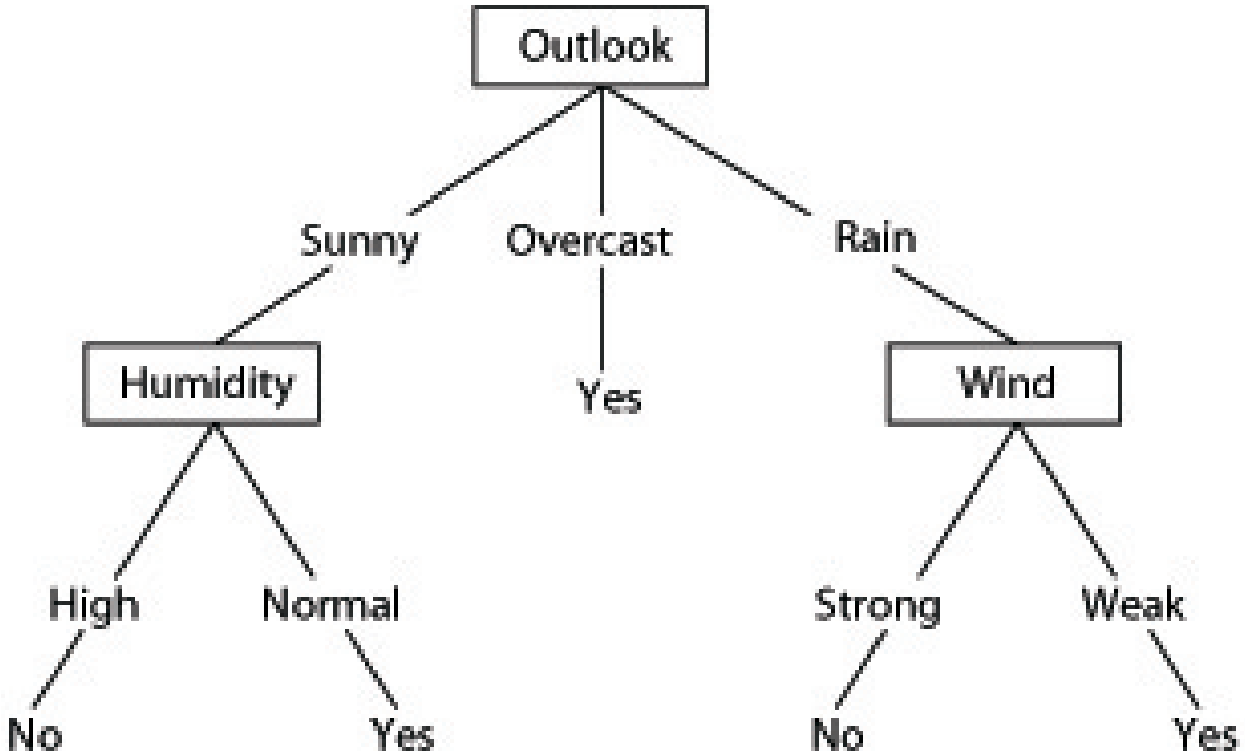
Rule Post-Pruning (Used in Algorithm C4.5)

What can we do when data are limited?

A possible method (many others exist):

1. Infer the decision tree allowing for overfitting
2. Convert the learned tree into an equivalent **set of rules**
3. Prune (generalize) each rule independently of others
4. Sort final rules by their estimated accuracy and use them in this order

Converting A Tree to Rules



A rule is generated for each leaf node (path from root to leaf).

Converting a Tree to Rules

R1: IF $(Outlook = Sunny) \wedge (Humidity = High)$
THEN $PlayTennis = No$

R2: IF $(Outlook = Sunny) \wedge (Humidity = Normal)$
THEN $PlayTennis = Yes$

...

- Then each rule is pruned by removing any antecedent whose removal does not worsen its estimated accuracy.
  - Example: prune $(Outlook = Sunny)$ or $(Humidity = high)$ from R1
- No pruning is performed if it reduces accuracy
- Accuracy estimation: use a validation set or a *statistical (pessimistic) test on the full set of examples*